TetraMem Take-Home Task: Systolic Array Implementation

Jing-Hua (Joseph) Chang
M.S. in Computer Engineering
University of California, San Diego
jic184@ucsd.edu

Abstract

This is a digital systolic array that executes MAC operations in a weight-stationary fashion. The inputs are signed 4-bit integers. The outputs are signed 9-bit integers. Weights are pre-loaded N (N=3) cycles before the activation comes in. Outputs are valid 2N cycles after the weight pre-loading. Finally, the systolic array is verified with a continuous random matrix stream, corner cases, and algebraic definition.

1 Architecture Overview

My thought is that high-performance and high-throughput is the core reasons for using a systolic array. Instead of merely passing two consecutive matrix multiplications, we should assume the systolic array receives a long consecutive matrix stream. This way, we can bring out the advantage of using the double-buffering technique.

Following the suggested template interface, the design is as follows. The entire matrix is first loaded into a local buffer inside *systolic_top*, then it will be scheduled to be fed into the systolic array in a diagonal fashion. This methodology relieves the burden of the compiler, which is friendly for integration.

The local buffers are also double-buffered to support multiple consecutive executions. The shift registers are used for systolic flow control logic, indicating when each inputs/outputs are valid. It is mentioned that two 4-bit inputs require a 9-bit partial sum. Consequently, I assume signed integers as input activations and weights for this project. The overflow does not occur under this configuration.

Please refer to the *systolic.html* file for an interactive illustration of the design. Hover the cursor over the block to see the corresponding RTL. Click to see a detailed RTL. This implementation follows the TetraMem take-home task description and course materials [1, 2, 3].

2 Verification Strategy

2.1 Architectural Corner Cases

Boundary Saturation: Validated signed maximum (+7), signed minimum (-8).

$$\begin{bmatrix} 7 & 7 & 7 \\ 7 & 7 & 7 \\ 7 & 7 & 7 \end{bmatrix} \times \begin{bmatrix} -8 & -8 & -8 \\ -8 & -8 & -8 \\ -8 & -8 & -8 \end{bmatrix} = \begin{bmatrix} -168 & -168 & -168 \\ -168 & -168 & -168 \\ -168 & -168 & -168 \end{bmatrix}$$

$$\begin{bmatrix} -8 & -8 & -8 \\ -8 & -8 & -8 \\ -8 & -8 & -8 \end{bmatrix} \times \begin{bmatrix} -8 & -8 & -8 \\ -8 & -8 & -8 \\ -8 & -8 & -8 \end{bmatrix} = \begin{bmatrix} 192 & 192 & 192 \\ 192 & 192 & 192 \\ 192 & 192 & 192 \end{bmatrix}$$

Preprint.

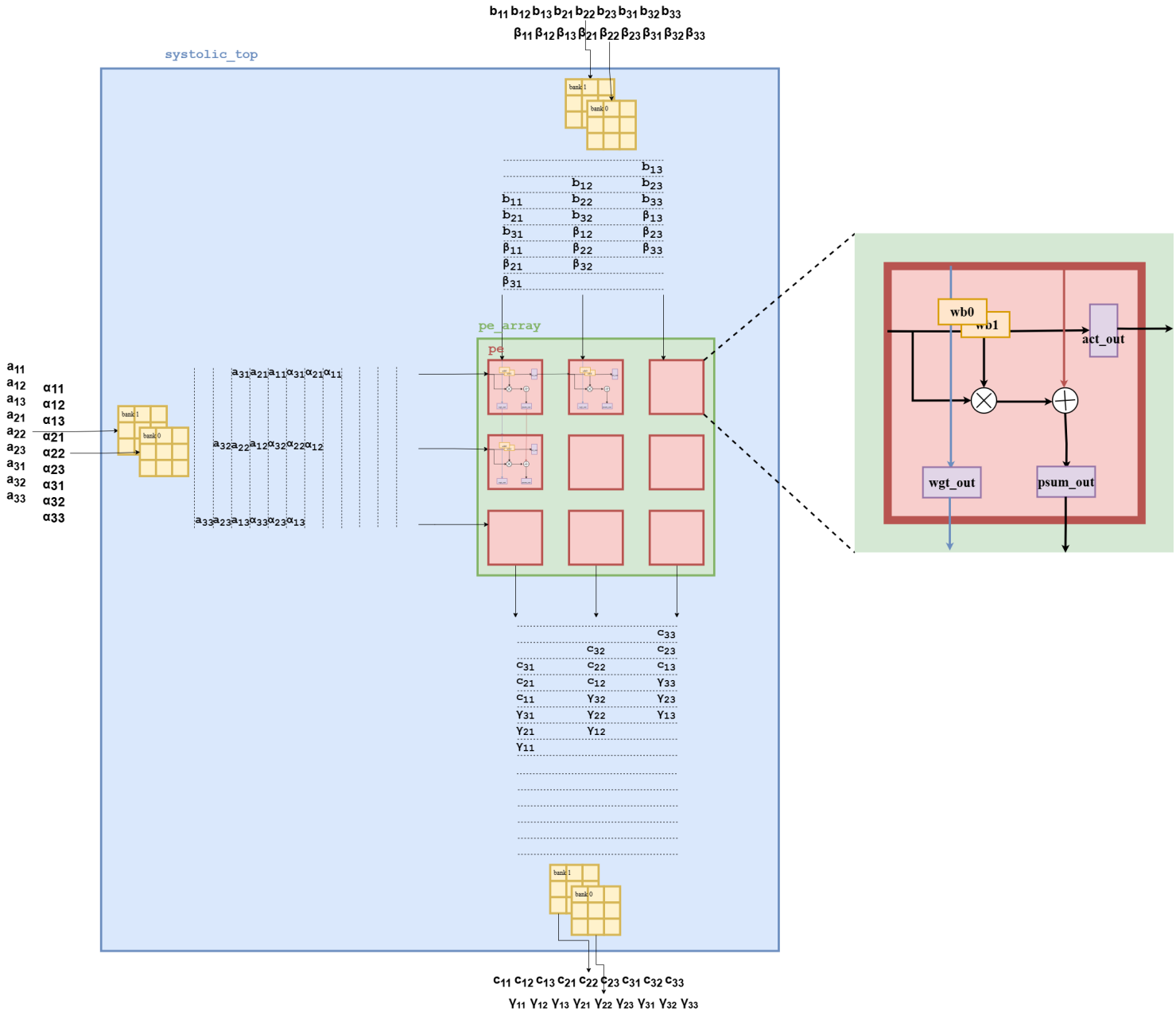

Figure 1: The design topology and data flow of the 3x3 systolic array.

Zero-State Integrity: Streamed all-zero matrices to guarantee that internal partial sums are completely cleared between back-to-back operations.

Bitwise Extremes: Processed all-one matrices to verify the behavior of the MAC units under uniform bit toggling.

2.2 Algebraic Proof Validation

Identity Property:

$$A \times I = A$$

$$A \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = A$$

Zero Property:

$$A \times 0 = 0$$

$$A \times \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Non-Commutativity:

$$A \times B \neq B \times A$$

Scalar Linearity:

$$(cA) \times B = cC$$

$$(2A) \times B = 2(A \times B)$$

2.3 Continuous Randomization

High-Volume Streaming: Successfully pipelined 54 randomized matrices, ensuring the double-buffering logic enables continuous executions. See Fig. 2:

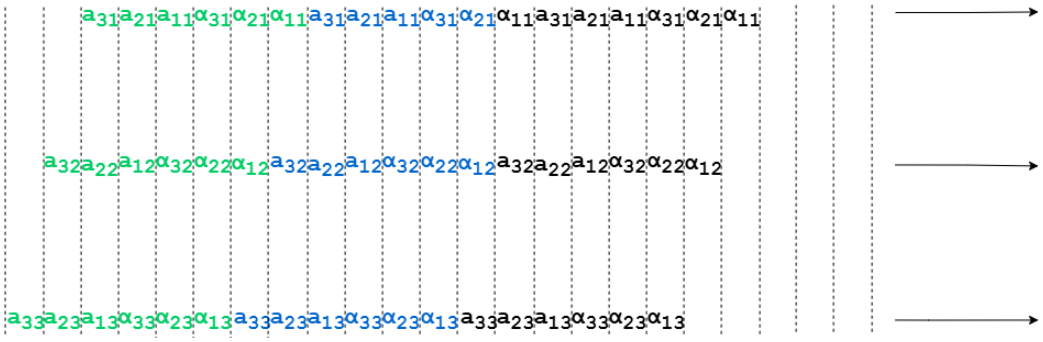

Figure 2: High-performance matrix stream.

3 Optimizations

High-Performance Matrix Stream: Additional pipeline registers are introduced for uninterrupted execution of the matrix stream. Since the two consecutive matrices are overlapped during diagonal data flow, we can not fetch the next matrix before the third cycle of the second matrix execution. As a result, the active buffer bank is first fed into a pipeline before entering the systolic array.

4 Results and Analysis

4.1 Waveform

We can see from the waveform that the systolic inputs and outputs are consistent with our design overview. Assuming the matrix is loaded at the 0th cycle, then the *output_valid* signal is raised at the 10th cycle. The value inside the output buffer and the bypassed value will be output as an entire array in the 10th cycle.

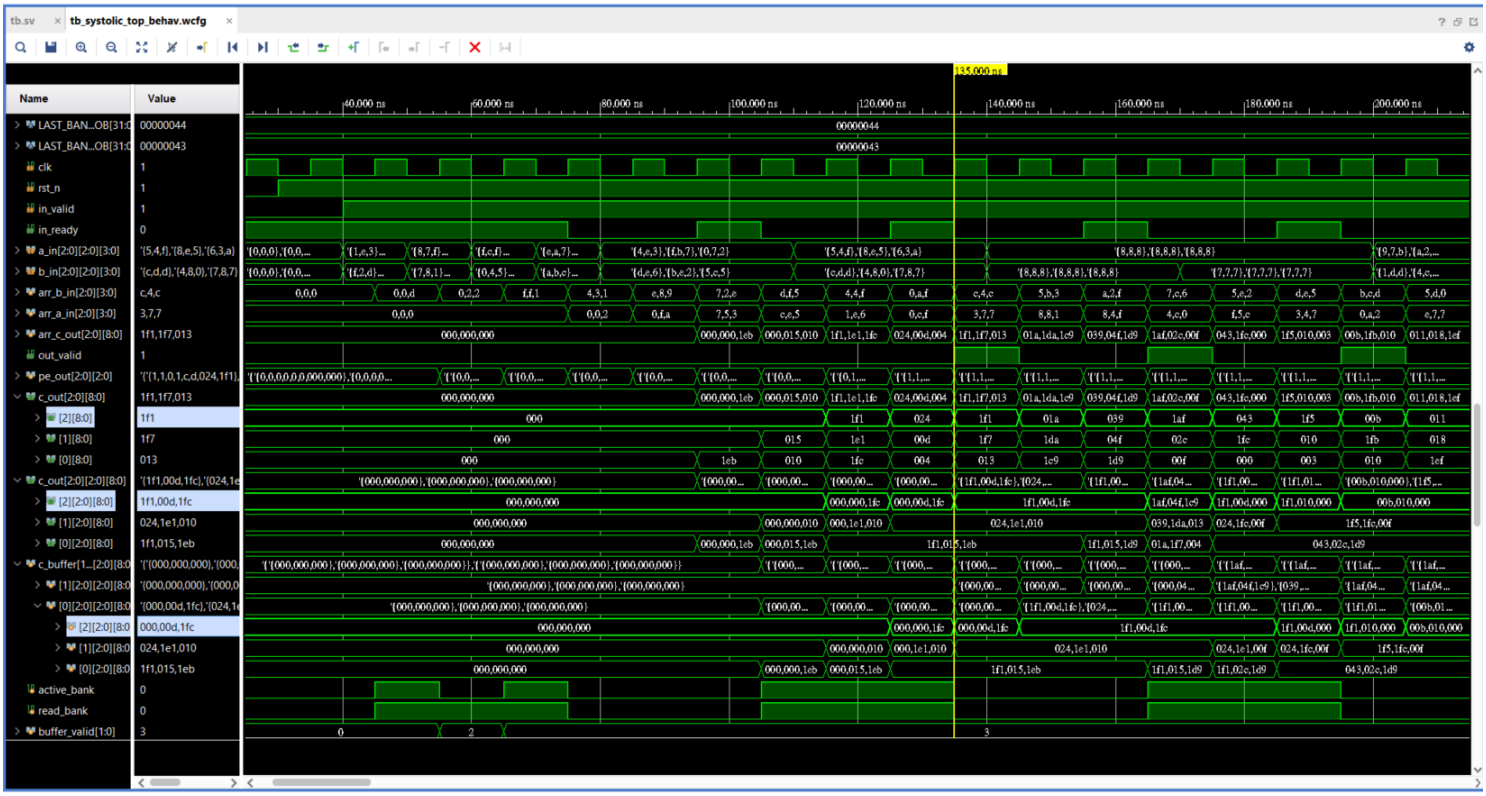

Figure 3: Waveform showing the start of a long matrix stream.

4.2 Console

The log file shows the testbench results, which specifically indicate the computation correctness and the visualization of each test case.

5 Tool Usage

The full codebase is available at `https://github.com/Josephood7/systolic-array-3x3`. The simulation is performed on Vivado 2023.2 (xcu200-fsgd2104-2-e). The block-diagram design topology is illustrated using draw.io. The systolic array is designed by Jing-Hua Chang using SystemVerilog. The testbench is fully generated by ChatGPT 5.6 Sol Extra High using detailed prompting and specification. This report is written in LaTeX, compiled on Overleaf.

References

[1] TetraMem. Systolic array implementation.

[2] Mingu Kang. Low-power vlsi implementation for machine learning. *University of California, San Diego*.

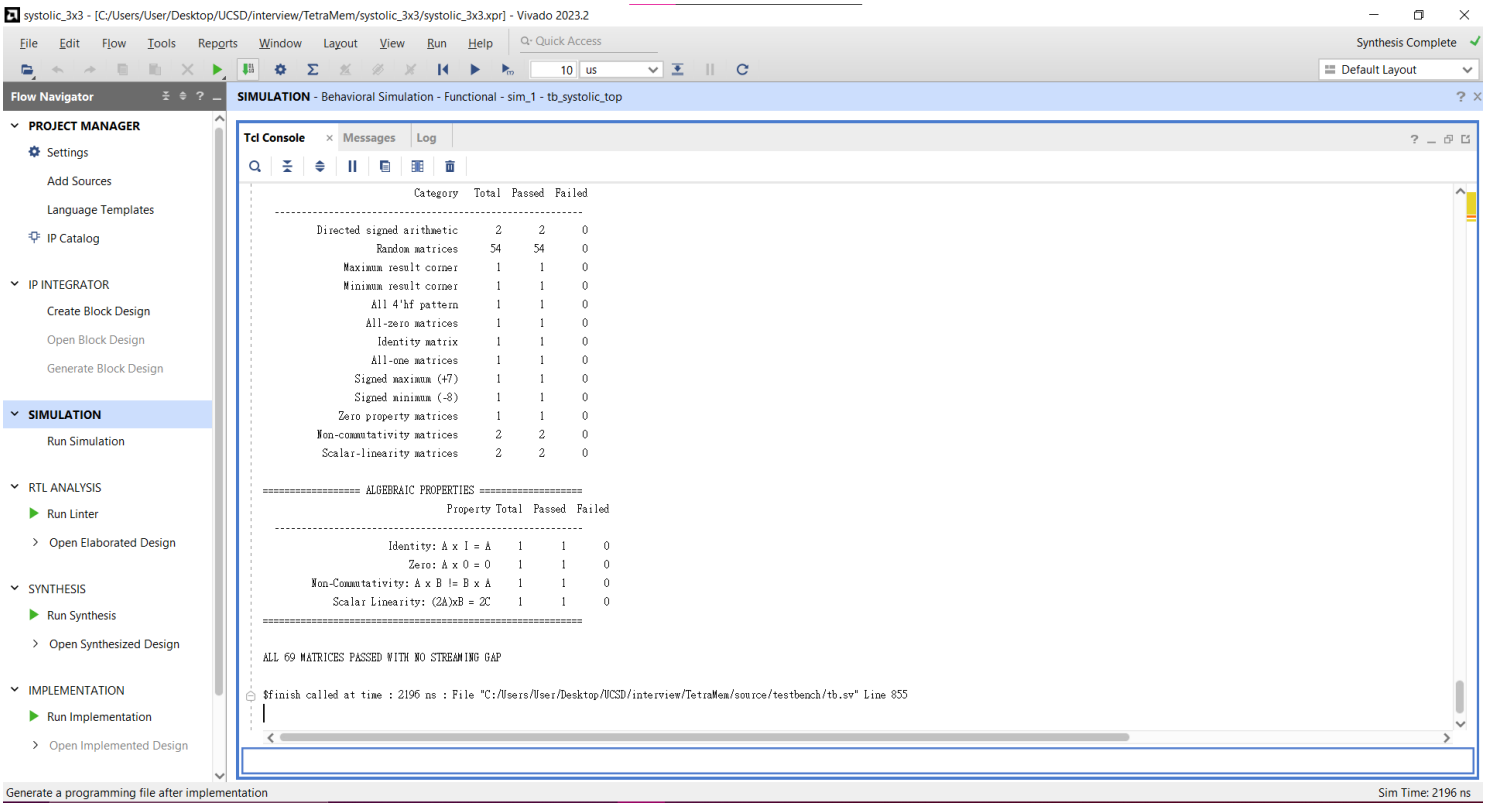

Figure 4: Final verification summary.

```
================================================================================================
TESTCASE 53: Random matrix
ATTRIBUTE   : Verifies general signed matrix multiplication with randomized 4-bit data.
------------------------------------------------------------------------------------------------
      Matrix A            Matrix B            Golden C=A*B          RTL C
  +----------------+  +----------------+  +----------------+  +----------------+
  |   -3   -4   -8 |  |    7   -6    6 |  |  -85  -26  -10 |  |  -85  -26  -10 |
  |    5   -3    0 |  |    6    1    6 |  |   17  -33   12 |  |   17  -33   12 |
  |    7   -4    6 |  |    5    5   -4 |  |   55  -16   -6 |  |   55  -16   -6 |
  +----------------+  +----------------+  +----------------+  +----------------+
RESULT      : PASS -- RTL output matches the golden matrix.
================================================================================================
```

Figure 5: Random matrix test case verification.

[3] Mingu Kang. Vlsi integrated circuits and systems design. *University of California, San Diego*.